

LINUX FUNDAMENTALS

Colorful handwritten-style lecture notes

A practical beginner course

From your first terminal command to managing a Linux host with confidence.

For learners: *students, new administrators, developers, cloud and DevOps beginners.*

LECTURE	WHAT YOU WILL LEARN
<i>Introduction</i>	<i>What Linux is, how its parts fit together, and where it is used.</i>
<i>Linux Basics</i>	<i>Terminal prompt, command syntax, navigation, and help.</i>
<i>Working in the Linux Shell</i>	<i>Bash, quoting, variables, pipes, redirection, and exit codes.</i>
<i>Linux Server and Clients</i>	<i>Client-server model, IP, DNS, ports, firewalls, and SSH.</i>
<i>Basic Server Administration</i>	<i>Users, packages, services, processes, logs, and health.</i>
<i>The Linux File System</i>	<i>Hierarchy, paths, permissions, search, storage, and archives.</i>
<i>Working with the Linux GUI</i>	<i>Desktop components and safe graphical access.</i>
<i>Working on a Linux Host</i>	<i>Inspect, edit, script, monitor, verify, and record.</i>
<i>Linux Basics Summary</i>	<i>Command recap, safe habits, labs, quiz, and answers.</i>

How to use these notes

Read the idea first, type each command in a disposable Linux lab, inspect the result, then try the practice task. Commands that change a system are labelled. Use a virtual machine or approved training host.

Course map and lab safety

A useful habit: observe first, make one change, then verify.

At the end of these lectures, you should be able to: *identify a Linux distribution; read a shell prompt; use paths and files; explain permissions; connect to a server; inspect services and logs; and perform basic administration safely.*

STAGE	QUESTION TO ASK	EXAMPLE
Observe	What system am I on?	<code>whoami</code> , <code>hostnamectl</code> , <code>df -h</code>
Understand	What does the command change?	<code>man command</code> , <code>command --help</code>
Act	Can I make one small change?	Create a lab file; restart only the lab service.
Verify	Did the result match my goal?	<code>Check_status</code> , output, logs, and exit code.

Safety rules for a beginner lab

- Practise in a virtual machine, container, or training server. Do not experiment on a work server.
- Read a command before pressing Enter. Be cautious with `sudo`, `rm`, `chmod -R`, `chown -R`, and commands copied from the internet.
- Never paste secrets, passwords, private keys, or production data into examples or public terminals.
- Before a risky change: check the target, take a backup or snapshot, use the smallest scope, and make a rollback plan.
- Linux commands are case-sensitive. A space can change the meaning of a command.

Course route

Introduction -> Linux basics -> shell -> clients and servers -> administration -> file system -> GUI -> work on a host -> summary and practice.

What is Linux?

Think of it as the engine and toolkit that let people and programs use a computer.

Linux is an open-source operating-system kernel. In everyday conversation, people also say Linux to mean a complete operating system built around that kernel, such as Ubuntu, Debian, Fedora, or Red Hat Enterprise Linux.

A simple computer picture

Hardware (CPU, memory, disk, network) -> **Linux kernel** -> **system tools and services** -> **shell or desktop** -> **your apps**

The kernel manages resources and gives programs controlled access to hardware. The shell and desktop are ways for people to ask the system to do work.

WORD	PLAIN MEANING	EXAMPLE
Kernel	Core that manages hardware, memory, processes, and devices.	Linux kernel
Distribution (distro)	A packaged Linux system with tools, installer, updates, and defaults.	Ubuntu, Debian, Fedora
Shell	A command interpreter that reads commands and starts programs.	Bash, Zsh
Terminal	A text window or connection that displays a shell session.	GNOME Terminal, SSH session
Service	A background program that provides a function.	sshd, web server, database

Where Linux is used

Cloud servers, supercomputers, Android devices, routers, containers, developer workstations, and embedded systems. It is popular when teams need automation, stability, choice, and control.

How the pieces start and work together

A short mental model for reading Linux documentation.

When a computer starts, firmware begins the boot process. A bootloader loads the Linux kernel. The kernel initializes hardware and starts the first system process. That process starts services and login options. You then use a terminal, a desktop, or a remote connection.

Boot sequence at a glance

Power on -> bootloader -> Linux kernel -> init system -> services -> login -> applications

Open source means the source code can be inspected and, under its licence, modified and redistributed. It does not mean every Linux product is free of cost, support, or commercial terms. Organizations may pay for tested releases, security updates, and technical support.

Choose a distribution for the task

For a first desktop lab, Ubuntu Desktop or Linux Mint is approachable. For server practice, Ubuntu Server or a supported enterprise Linux distribution is common. Learn the commands and concepts first; package-manager names and some paths vary by distribution.

Remember

Linux is not one single desktop or one single command set. The kernel is shared by many distributions, while installers, desktops, package tools, and default settings can differ.

Your first terminal session

A terminal accepts text; a shell interprets what you type.

Reading the prompt

```
vishwa@lab-host:~$ pwd
/home/vishwa
vishwa@lab-host:~$
```

PROMPT PART	MEANING
<i>vishwa</i>	<i>Current user</i>
<i>lab-host</i>	<i>Computer name, also called hostname</i>
<i>~</i>	<i>Current directory; ~ means this user's home directory</i>
<i>\$</i>	<i>Usually a regular-user prompt; # often indicates root</i>

Command shape: *command [options] [arguments]. Options change behavior; arguments identify what to act on. Spaces separate parts. Read the manual or built-in help when unsure.*

One command, broken into parts

```
ls -la /var/log
ls command
-la options: long format and include hidden names
/var/log argument: directory to list
```

Core navigation commands

pwd = show current directory | ls = list names | cd = change directory | whoami = show current user | date = show date and time | uname -a = show kernel and host details.

Navigate without getting lost

Paths tell Linux which place you mean.

COMMAND	WHAT IT DOES	EXAMPLE
<code>pwd</code>	Print the current path.	<code>pwd</code>
<code>ls -lah</code>	List files, including hidden ones, in readable size units.	<code>ls -lah</code>
<code>cd /etc</code>	Move to an absolute path.	<code>cd /etc</code>
<code>cd ..</code>	Move to the parent directory.	<code>cd ..</code>
<code>cd ~</code>	Go to your home directory.	<code>cd ~</code>
<code>mkdir practice</code>	Create a directory.	<code>mkdir practice</code>
<code>touch notes.txt</code>	Create an empty file, or update its timestamp.	<code>touch notes.txt</code>
<code>man ls</code>	Open the manual page; press <code>q</code> to quit.	<code>man ls</code>

Absolute and relative paths

Absolute: starts at `/`, the top of the file tree. Example: `/home/vishwa/notes.txt`

Relative: starts from where you are now. Example: `../shared/notes.txt`

Special shortcuts: `.` means this directory; `..` means parent; `~` means your home directory.

Try it: move around and review recent commands

```
cd ~/practice
pwd
ls -la
cd ..
history | tail
```

Help is part of the workflow

Try `man` command, `command --help`, or `help` for a Bash built-in. In a manual page, search with `/word`, move with the arrow keys, and exit with `q`.

How Bash reads a command

The shell expands names and symbols, then starts a program.

A shell is a command interpreter. Bash is a widely used shell. Your terminal is the screen or connection where you interact with that shell. A shell prompt is ready for a command; after you press Enter, it displays output and returns to a prompt.

FEATURE	EXAMPLE	WHAT HAPPENS
Variable	<code>name='Linux learner'; echo "\$name"</code>	Stores text, then prints it. Quotes preserve spaces.
Home shortcut	<code>cd ~/Documents</code>	~ expands to the user's home path.
Wildcard	<code>ls *.log</code>	* matches zero or more characters.
Command substitution	<code>today=\$(date +%F)</code>	Runs date and stores its output.
Environment variable	<code>echo "\$PATH"</code>	Shows folders searched for commands.

A few safe shell examples

```
greeting='Hello, Linux learner'
echo "$greeting"
echo "$HOME"
echo "$PATH"
```

Why quote variable values?

Use double quotes around variables such as `"$name"` so spaces stay inside one argument and wildcard characters are not expanded unexpectedly. Single quotes keep text literal; double quotes still allow variable expansion.

Combine small tools

Pipes and redirection pass data between commands.

SYMBOL	MEANING	EXAMPLE
	Send one command's standard output to another command.	<code>ps aux grep ssh</code>
>	Write output to a file, replacing its old contents.	<code>date > today.txt</code>
>>	Append output to a file.	<code>echo done >> today.txt</code>
2>	Send error output to a file.	<code>tool 2> errors.txt</code>
&&	Run the next command only if the first succeeds.	<code>mkdir demo && cd demo</code>
	Run the next command if the first fails.	<code>test -f app.conf echo missing</code>

A pipeline example

```
cat access.log | grep ' 404 ' | wc -l
# count log lines containing the text 404
grep -n 'ERROR' app.log > errors.txt
# save matching lines, with line numbers, in errors.txt
```

Exit status

A command returns a status code when it finishes. 0 usually means success; a non-zero value usually means an error or a special condition. Check the last command with `echo $?`. For a pipeline, Bash normally reports the last command's status unless `pipefail` is enabled.

Safe command habits

Use `ls` or `pwd` before acting on a path. Preview matches before deleting. Avoid running an unfamiliar command with `sudo`. Redirection with `>` replaces a file, so use `>>` when you mean append.

How a client reaches a service

A server is a role a computer or program performs.

A client requests a service. **A server** listens for requests and responds. The same computer can be a client for one service and a server for another. A web browser is a client; a web server listens for web requests.

CONCEPT	PLAIN MEANING	EXAMPLE
IP address	Network address used to reach a host.	10.0.2.15
DNS	Looks up a name and returns an address.	training.example
Port	Number that identifies a service endpoint on a host.	SSH commonly uses TCP 22
Firewall	Rules that allow or block network traffic.	Allow SSH only from a trusted network
Protocol	Rules for communication.	SSH, HTTPS, DNS

Simple request path

Browser or SSH client -> DNS lookup (if using a name) -> network route -> firewall check -> server port -> service response.

A port number alone does not prove that a service is safe or reachable. A service must be running, listening on the correct interface, and allowed by host and network firewalls.

Connect to a Linux server with SSH

SSH provides an encrypted remote terminal connection.

SSH: connect, use the terminal, then disconnect

```
ssh username@server.example
ssh -i ~/.ssh/lab_key username@203.0.113.10
exit
```

- **username:** account on the remote host.
- **server.example:** DNS name; an IP address can be used instead.
- **-i:** optional identity file containing your private key. Keep that key private.
- On first connection, SSH may ask whether to trust the host key. Verify the fingerprint through a trusted channel before accepting it on an important system.
- The remote shell prompt is not your local computer. Check `hostname` and `whoami` before making changes.

First checks after connecting

```
hostname
whoami
pwd
uptime
exit
```

SSH safety

Prefer key-based access, protect private keys with a passphrase, use named accounts, and restrict network access to approved sources. Never share a private key or disable host-key checks just to silence a warning.

Users, groups, packages

Use the least privilege needed for the task.

A user account gives a person or service an identity. A group lets an administrator apply shared access.

root is the all-powerful administrator account. **sudo** lets an approved user run a particular command with elevated privileges, usually after authentication.

Inspect your identity; elevate only a needed command

```
whoami
id
groups
sudo command
```

TASK	UBUNTU / DEBIAN EXAMPLE	FEDORA / RHEL EXAMPLE
<i>Refresh package metadata</i>	<code>sudo apt update</code>	<code>sudo dnf check-update</code>
<i>Install a package</i>	<code>sudo apt install tree</code>	<code>sudo dnf install tree</code>
<i>Upgrade packages</i>	<code>sudo apt upgrade</code>	<code>sudo dnf upgrade</code>
<i>Find package details</i>	<code>apt show tree</code>	<code>dnf info tree</code>

Package manager vocabulary

A repository is a trusted software catalogue. Refreshing metadata checks what versions are available. Installing adds a package and dependencies. Upgrading replaces installed packages with newer versions. Check your organization's change policy before updating a server.

Services, processes, and logs

A service runs in the background; a process is a running program.

Service checks (service name may be sshd on some systems)

```
systemctl status ssh
sudo systemctl start ssh
sudo systemctl stop ssh
sudo systemctl enable --now ssh
journalctl -u ssh --since today
```

COMMAND	USE
<code>ps aux</code>	<i>Snapshot of running processes.</i>
<code>top</code>	<i>Live view of CPU and memory use; press q to quit.</i>
<code>pgrep -a name</code>	<i>Find process IDs and matching command lines.</i>
<code>kill PID</code>	<i>Ask a process to stop; use only for a process you own or administer.</i>
<code>journalctl -xe</code>	<i>Review recent system journal messages.</i>
<code>df -h</code>	<i>Check free space on mounted file systems.</i>

Admin order of operations

Check status -> read logs -> identify the exact service and host -> make one approved change -> check status and logs again. Do not kill a process just because its name looks unfamiliar.

Check host health before a change

Measure first so you can compare after.

QUESTION	COMMAND	WHAT TO NOTICE
<i>What host is this?</i>	<i>hostnamectl</i>	<i>Host name, operating system, kernel</i>
<i>How long has it run?</i>	<i>uptime</i>	<i>Uptime and load averages</i>
<i>Is memory tight?</i>	<i>free -h</i>	<i>Used, available, and swap</i>
<i>Is a disk nearly full?</i>	<i>df -h</i>	<i>Use percent and mount point</i>
<i>Which service listens?</i>	<i>sudo ss -tulpn</i>	<i>Address, port, and owning process</i>
<i>Can I reach a name?</i>	<i>getent hosts example.com</i>	<i>Name resolution result</i>
<i>Can I reach a host?</i>	<i>ping -c 4 server.example</i>	<i>Basic network reachability; may be blocked</i>

Updates need a plan

On a personal lab, install security and package updates regularly. On a managed server, follow its maintenance window, backup and rollback procedure, and application owner approval. A kernel update may require a reboot; check the vendor or platform guidance.

If something goes wrong

Capture the exact error and time. Check service status, recent logs, disk space, and recent changes. Avoid repeated restarts until you know what the service is doing and what depends on it.

Linux folders at a glance

Everything is arranged beneath one starting point: /

PATH	COMMON PURPOSE
/	Top of the directory tree.
/home	Personal home folders for regular users.
/root	Home folder for the root administrator.
/etc	System and service configuration files.
/var	Frequently changing data such as logs, caches, and queues.
/var/log	Many system and application log files.
/tmp	Temporary files; cleanup rules vary.
/usr	Many installed programs, libraries, and shared data.
/opt	Optional or third-party application software.
/dev and /proc	Device entries and process/kernel information exposed by the system.

File names and hidden names

Names beginning with a dot, such as `.bashrc`, are hidden from a plain `ls` listing. Use `ls -la` to see them. Linux distinguishes upper and lower case: `Notes.txt` and `notes.txt` can be different files.

Read permissions: *rwX*

Permissions are set separately for owner, group, and others.

LETTER	ON A FILE	ON A DIRECTORY
<i>r</i> - read	Read file contents.	List names inside the directory.
<i>w</i> - write	Change file contents.	Create, rename, or remove entries inside it, subject to other controls.
<i>X</i> - execute	Run the file as a program, if it is executable.	Traverse/search the directory to access known names inside it.

Read an `ls -l` permission string

```
-rw-r----- 1 maya analysts 240 Sep 28 10:30 report.txt
owner: maya group: analysts
owner rw- group r-- others ---
```

Numeric permissions

Read = 4, write = 2, execute = 1. Add the values for each class: 7 means *rwX*, 6 means *rw-*, 5 means *r-X*, and 0 means no access. Example 640 means owner *rw-*, group *r--*, others *---*.

Change permissions or ownership only when authorized

```
chmod 640 report.txt
chmod u+x deploy.sh
chown maya:analysts report.txt
umask
```

Find, inspect, and package files

Choose a narrow search path before scanning a large system.

COMMAND	PURPOSE	EXAMPLE
<code>cat</code>	Print a small text file.	<code>cat settings.conf</code>
<code>less</code>	Read a longer file page by page.	<code>less /var/log/syslog</code>
<code>head / tail</code>	Show the start or end of a file.	<code>tail -n 30 app.log</code>
<code>grep</code>	Search text lines.	<code>grep -n 'ERROR' app.log</code>
<code>find</code>	Search by path and name.	<code>find ~/practice -name '*.txt'</code>
<code>cp / mv</code>	Copy or move/rename files.	<code>cp notes.txt notes.bak</code>
<code>rm</code>	Remove files. There is normally no trash recovery.	<code>rm lab.tmp</code>
<code>tar</code>	Bundle files for transfer or backup.	<code>tar -czf lab.tar.gz practice/</code>

Search and check storage

```
find ~/practice -type f -name '*.log'
grep -Rni 'failed' ~/practice
du -sh ~/practice
df -h
```

Before deleting

First list the exact matches. Confirm the path and whether it is a file or directory. Avoid broad wildcards and recursive deletion until you understand every target. On shared systems, backups and retention rules matter.

Working with a desktop

A graphical desktop and a command shell are two ways to work with the same system.

A Linux desktop commonly includes a **desktop environment** (panels, menus, settings), a **window manager** (windows and placement), a **display manager** (login screen), and applications such as a file manager and terminal. GNOME, KDE Plasma, and Xfce are examples of desktop environments.

GUI TASK	WHAT TO DO	CLI EQUIVALENT
Browse folders	Open the file manager and use the location bar.	<code>pwd, ls, cd</code>
Edit text	Open a text editor and save a plain-text file.	<code>nano file.txt</code>
Install apps	Use the distro's software centre.	<code>apt</code> or <code>dnf</code>
Inspect network	Open network settings.	<code>ip addr, ip route</code>
Change display	Use display settings.	Often managed by desktop tools

Desktop or server?

Many servers run without a graphical desktop to reduce resource use and exposed software. A server can still be administered remotely through SSH. Installing a GUI on a production server should be a deliberate, supported decision.

Remote desktop caution

Remote GUI tools can be useful in a lab, but expose powerful access. Keep them behind approved network controls, use strong authentication, and disable them when not needed. Do not expose VNC or RDP directly to the public internet.

A repeatable host workflow

Know where you are before you change anything.

STEP	ACTION	USEFUL CHECK
1. Connect	Use the approved local console or SSH route.	<code>hostname; whoami</code>
2. Inspect	Understand OS, directory, and available space.	<code>cat /etc/os-release; pwd; df -h</code>
3. Locate	Find the correct file or service.	<code>ls -l; systemctl status name</code>
4. Read	Review help, config, and logs before editing.	<code>man; less; journalctl</code>
5. Change	Make one small, authorized change.	Use an editor; keep a backup.
6. Verify	Test the result and inspect logs.	status, output, application check
7. Record	Write down what changed and how to undo it.	Change ticket or lab notes

A safe first-minute checklist

```
cat /etc/os-release
hostname
whoami
pwd
df -h
systemctl --failed
```

Edit a text file

For a terminal beginner, nano is approachable: `nano filename`, edit text, `Ctrl+O` to write the file, `Enter` to confirm the name, and `Ctrl+X` to exit. On a system without nano, use the editor approved for that host.

Run a small script

A script is a text file of commands that the shell can run.

hello.sh

```
#!/usr/bin/env bash
set -e
echo 'Hello from Linux'
date
hostname
```

Create, mark executable, and run in your lab

```
nano hello.sh
chmod u+x hello.sh
./hello.sh
bash hello.sh
```

The first line is a **shebang**: it names the interpreter. The `./` prefix means run the file from the current directory. A file in the current directory is not usually found by typing only its name; this protects against accidentally running an untrusted local file.

Environment variables

Environment variables pass settings to programs. Use `printenv` or `echo "$HOME"` to inspect common values. Set a temporary variable with `NAME=value` for the current shell; prefix a command with `NAME=value` to pass it only to that command. Do not put passwords or tokens in scripts or shell history.

Logs help explain behavior

Use `journalctl -u service-name` to inspect a systemd service. Application logs may also be stored under `/var/log` or an application-specific path. Log locations differ; follow the distribution and application documentation.

Mini scenario: a web service will not start

Practice a calm sequence in a disposable lab.

ORDER	CHECK	WHY
1	<code>hostname; date</code>	Confirm the host and time.
2	<code>systemctl status nginx</code>	Check whether the service is active and read the immediate error.
3	<code>journalctl -u nginx --since today</code>	Review recent service messages.
4	<code>sudo ss -tulpn</code>	Check whether the expected port is already in use.
5	<code>df -h; free -h</code>	Check for obvious disk or memory pressure.
6	Make one approved correction, then restart if appropriate.	Avoid changing several things at once.
7	<code>systemctl status nginx</code> ; test the application.	Verify service health and user-visible behavior.

Think aloud

What changed most recently? What does the exact error say? Which log line supports your theory? What is the smallest safe test? How will you reverse the change? This is the same disciplined approach used by production administrators.

Backups and restore

A backup is useful only if it can be restored. For a lab, copy a configuration file before editing it, keep the copy permission-safe, and verify the restore path. On real systems, follow the organization's backup process rather than improvising.

Commands to keep close

Start with navigation, identity, inspection, and help.

JOB	COMMANDS
<i>Where am I?</i>	<i>pwd, hostname, whoami, id</i>
<i>Look around</i>	<i>ls -lah, tree (if installed), find</i>
<i>Move around</i>	<i>cd /path, cd .., cd ~</i>
<i>Read files</i>	<i>cat, less, head, tail</i>
<i>Search text</i>	<i>grep -n, grep -R</i>
<i>Create and copy</i>	<i>mkdir, touch, cp, mv</i>
<i>Permissions</i>	<i>ls -l, chmod, chown</i>
<i>Services and logs</i>	<i>systemctl status, journalctl -u</i>
<i>Processes and health</i>	<i>ps aux, top, free -h, df -h, uptime</i>
<i>Network</i>	<i>ip addr, ip route, ss -tulpn, getent hosts</i>
<i>Remote session</i>	<i>ssh user@host, exit</i>

Command pattern to remember

command [options] [target]. Read help, use a narrow target, check the result. Use sudo only when the task requires administrator privileges.

Permissions and service commands

Check before, change once, check again.

Common forms

```
ls -l file.txt # inspect owner and permissions
chmod 640 file.txt # owner rw, group r, others none
chmod u+x script.sh # add execute permission for owner
chown user:group file.txt # change ownership when authorized
systemctl status SERVICE # inspect a service
sudo systemctl restart SERVICE # restart only when planned
journalctl -u SERVICE # view service logs
```

IF YOU SEE	FIRST THOUGHT
<i>Permission denied</i>	<i>Check the path and permissions; do not immediately add sudo.</i>
<i>Command not found</i>	<i>Check spelling, package installation, and PATH.</i>
<i>No such file or directory</i>	<i>Check current directory, capitalization, and path.</i>
<i>Disk full</i>	<i>Check df -h and identify what is using space before deleting.</i>
<i>Service failed</i>	<i>Read systemctl status and journalctl for the exact service.</i>

Red-flag command reminder

sudo rm -rf, recursive chmod/chown, and commands pasted from unknown sources can cause broad damage. Verify every path and option first. In a class, demonstrate destructive commands only with a safe explanation and disposable targets.

Linux concepts in one minute

Say it in your own words before moving to the lab.

- **Linux** is commonly used to mean an operating system built around the Linux kernel.
- A **distribution** packages the kernel, tools, installer, repositories, and defaults.
- A **shell** interprets commands; a **terminal** is where a text session appears.
- A **server** provides a service; a **client** requests it.
- **SSH** gives an encrypted remote terminal session.
- The **file system** is a tree rooted at /; paths locate files and directories.
- Permissions control read, write, and execute access for owner, group, and others.
- **sudo** elevates a specific approved command; root has broad administrative power.
- Use **systemctl** to inspect services and **journalctl** to read system logs.
- Observe, understand, act carefully, verify, and record.

The best beginner habit

When you do not know what a command will do, stop and read its help page. A few seconds of checking is faster than recovering from a preventable mistake.

Lab 1: explore your own workspace

No administrator privileges required.

TASK	DO THIS
1. Identify yourself	Run <code>whoami</code> and <code>id</code> . Write down the user and groups.
2. Inspect location	Run <code>pwd</code> , then <code>ls -lah</code> .
3. Make a lab folder	Run <code>mkdir -p ~/linux-practice/week1</code> .
4. Create notes	Run <code>cd ~/linux-practice/week1</code> then <code>nano notes.txt</code> . Add one sentence and save.
5. Inspect the file	Run <code>ls -l notes.txt</code> , <code>cat notes.txt</code> , and file <code>notes.txt</code> .
6. Copy a backup	Run <code>cp notes.txt notes.bak</code> , then compare with <code>diff notes.txt notes.bak</code> .
7. Find it	Run <code>find ~/linux-practice -name 'notes*'</code> .

Expected learning

You navigated with paths, created a directory and file, read metadata, copied a backup, and searched a narrow location. If you get an error, read it carefully and check `pwd` before repeating the command.

Lab 2: permissions and host checks

Use a VM or approved training host.

Try each line and explain the result

```
cd ~/linux-practice/week1
printf '#!/usr/bin/env bash\nnecho hello\n' > hello.sh
ls -l hello.sh
chmod u+x hello.sh
./hello.sh
stat hello.sh
```

- What changed after `chmod u+x`?
- What does the first character in `ls -l` show?
- Run `hostname`, `uptime`, `free -h`, `df -h`, and `ip addr`. Which ones inspect the host?
- If `systemctl` is available, inspect a familiar service with `systemctl status`. Do not stop or restart it in a shared environment.

Instructor note

The `printf` example creates a small script. If typed into a shell, the quoted content contains a newline escape. Learners can instead create the file with `nano` to practise editing.

12 quick questions

Try without looking at the answer key.

- 1. What is the difference between a shell and a terminal?
- 2. What does `/` represent in a Linux path?
- 3. Which command prints the current directory?
- 4. What does `~` usually mean?
- 5. What does the pipe symbol `|` do?
- 6. In `chmod 640` file, what access does the group receive?
- 7. What is `sudo` used for?
- 8. Which command inspects a `systemd` service?
- 9. Which command shows a service's journal logs?
- 10. What is `SSH` commonly used for?
- 11. Does a server have to be a separate physical machine?
- 12. Name two checks to make before deleting a file.

Bonus discussion

A service is reported as unavailable. Give a calm first three checks and explain what each one tells you.

Answer key and glossary

Use the answers to find any topic to review.

- 1. A terminal is the interface that displays a session; a shell reads and interprets commands.
- 2. / is the root of the directory tree.
- 3. `pwd`.
- 4. The current user's home directory.
- 5. It sends standard output from one command to another command's input.
- 6. Read only (`r--`).
- 7. To run an approved command with elevated privileges.
- 8. `systemctl status service-name`.
- 9. `journalctl -u service-name`.
- 10. Encrypted remote login and command-line administration.
- 11. No. It can be a physical machine, virtual machine, or a program providing a service.
- 12. Confirm the exact path and target; inspect matching files; check backup/retention needs.

TERM	SHORT DEFINITION
CLI	Command-line interface; work by entering text commands.
GUI	Graphical user interface; windows, icons, menus, and pointer.
PID	Process identifier assigned to a running process.
Repository	A software source from which a package manager gets packages.
Log	Timestamped record of system or application events.
Root user	Administrator account with broad system privileges.

You have completed the lecture set

Keep practising in a safe lab. Revisit paths, permissions, SSH, services, and logs until you can explain what each command does before you run it.

LEARN BY DOING

Observe -> Understand -> Act -> Verify -> Record

Your Linux checklist

- *I know which host and user I am working as.*
- *I can explain the command and target before running it.*
- *I check permissions before reaching for sudo.*
- *I inspect status and logs after a change.*
- *I practise destructive operations only in a safe lab.*

Keep this notebook beside your terminal.